Overview

REMEDIATION REPORT

for

MEGAQUAGGA PUBLISHING

Version 1.0 (9/23/2025)

Prepared by **Brandon Cavalier**

Presented to **MEGAQUAGGA PUBLISHING**

DISCLAIMER: This document contains proprietary and confidential information. 0x2A SECURITY guarantees the confidentiality of the information and any other material that has or will be disclosed during the performance of this service. A mutual non-disclosure agreement of confidential information is assumed between 0x2A SECURITY and MEGAQUAGGA PUBLISHING, limiting the use of confidential information exclusively for purposes associated with properly delivering this service.

EXECUTIVE SUMMARY

Our recent security assessment identified a critical vulnerability in our web application that could have allowed an attacker to take complete control of our systems. To address this, we have implemented a new, layered security architecture that places a "secure receptionist" in front of our website. This receptionist, known as a reverse proxy, now inspects all incoming web traffic before it ever reaches our website, ensuring that malicious requests are stopped and our web server is protected.

This new architecture also upgrades our website from an unencrypted connection to a secure, encrypted one. This means any information exchanged with our site is now protected from being intercepted, safeguarding our data and our customers' trust. By taking these actions, we have not only fixed the immediate vulnerability but have also created a more resilient and secure foundation that will protect our systems and data from future attacks.

🦾 Actions Taken

ACTIONS TAKEN

This section outlines the actions taken on which assets to remediate which vulnerabilities.

Vulnerability	Remediation Solution
WordPress is vulnerable to web application attacks	enabled ModSecurity WAF and reverse proxy
Unencrypted web traffic	implemented SSL offloading on the reverse proxy

Remediation Solution #1: **Reverse Proxy and WAF**

The first part of the remediation was to protect the web server from malicious requests at the application layer.

1.Deploy a Reverse Proxy: An Apache web server was deployed at 10.20.30.220 to act as a reverse proxy. This was a strategic architectural change to introduce a new security layer. The proxy was configured to be the sole entry point for all web traffic, protecting the original web server from direct exposure.

2.Configure the WAF: The Apache server was then configured with a Web Application Firewall (WAF). This was a configuration change that specifically inspects incoming web requests. The WAF was given a custom rule to look for the malicious swp_url parameter associated with the RFI exploit and block any requests containing it.

3.Modify the Firewall Rule: The NAT rule on the pfSense firewall was changed as a configuration update. This change redirected all incoming HTTP traffic from its original destination to the new reverse proxy, ensuring that all traffic is inspected by the WAF before reaching the web server.

Remediation Solution #2: **SSL offloading**

1. Configure the HTTPS Virtual Host: A new configuration file, 000-default-ssl.conf, was created on the Apache reverse proxy. This configuration change enabled the proxy to handle encrypted HTTPS traffic.

2.Install a Certificate: A self-signed SSL certificate was installed on the Apache reverse proxy. This allows the server to establish a secure, encrypted connection with a user's web browser.

3.Modify the Firewall for HTTPS Traffic: The pfSense firewall's NAT rule was again updated, this time with a configuration change to allow HTTPS traffic on port 443. This ensured that secure connections could reach the reverse proxy and that the firewall would no longer drop this traffic.

✅ Evidence of Remediation

EVIDENCE OF REMEDIATION

This section documents evidence of the remediations being implemented. This evidence may take the form of details from packet captures (for example, alterations in traffic flow or packet headers), ModSecurity logs appearing in the Graylog SIEM, or output from command-line tools. For each remediation, we provide evidence of the original state as well as the current altered state.

Reverse proxy

These are alterations to MegaQuagga's environment associated with implementing the reverse proxy server responsible for implementing our two remediations (WAF & SSL offloading).

Item	**Original Vulnerable State**	**Current Remediated State**
Network Traffic Flow	Traffic was routed directly to the web server's private IP (10.20.30.130).	Traffic is now routed to the reverse proxy's IP (10.20.30.220).
Web Server Exposure	The web server was directly exposed to the internet via the firewall.	The web server is now hidden behind the reverse proxy.

Web Application Firewall (WAF)

These are alterations to MegaQuagga's environment associated with implementing the WAF remediation.

Item	**Original Vulnerable State**	**Current Remediated State**
Application Layer Defense	There was no defense to inspect incoming traffic at the application layer.	The reverse proxy now acts as a WAF, inspecting and filtering all traffic.
Vulnerability Status	The Social Warfare plugin was able to perform Remote File Inclusion (RFI) attacks.	The WAF has a rule to block requests when RFI meets a high point threshold. Blocking and mitigating the RFI vulnerability.
Network Traffic Flow	Malicious traffic was forwarded directly to the web server.(10.20.30.130)	Malicious traffic is now blocked by the WAF, (192.168.100.2) preventing it from ever reaching the web server.

SSL Offloading

These are alterations to MegaQuagga's environment associated with implementing the WAF remediation.

Item	Original Vulnerable State	Current Remediated State
Website Connection	The website was unencrypted, using HTTP on port 80.	The website is now encrypted, using HTTPS on port 443.
pfSense Firewall Rule	The firewall rule allowed traffic on port 80 to pass through.	The firewall rule now explicitly allows HTTPS traffic on port 443 to pass.
Apache Server Configuration	No configuration was in place to handle HTTPS traffic.	A new VirtualHost was created with a self-signed certificate to handle HTTPS traffic.
Traffic Redirection	Users could access the site via plain HTTP without a redirect.	All HTTP requests are now automatically redirected to HTTPS.
SSL Certificate	No SSL certificate was present on the server.	A self-signed certificate (megaquagga.crt and megaquagga.key) is in place.

🩺 Recommendations

RECOMMENDATIONS

This section outlines recommendations for continued monitoring and next steps for future remediations to further strengthen MegaQuagga's security posture.

Recommendation #1:**Log Analysis:**

The next step is to actively monitor these logs for signs of suspicious activity, such as repeated 403 Forbidden errors, which could indicate an attacker is testing your WAF rules, or unusual traffic patterns that might signal a new type of attack.

Recommendation #2: **Implement a Trusted SSL Certificate**

The current self-signed certificate is functional but causes a security warning for users. A major next step is to obtain a trusted SSL certificate from a Certificate Authority (CA) like Let's Encrypt. This would eliminate the "scare screen" and establish trust with all modern web browsers.

Recommendation #3: **Automate Updates and Patching:**

The original vulnerability was due to an outdated plugin. To prevent this from happening again, there should be an automated process for applying security patches and updates to the WordPress installation and its plugins. This is a proactive measure that will keep the site secure against newly discovered vulnerabilities.

📍 Conclusion

CONCLUSION

I implemented a reverse proxy with a Web Application Firewall (WAF) and enabled SSL offloading to create a layered security defense. This addressed a critical vulnerability that could have allowed an attacker to take control of our web server and mitigated the risk of transmitting unencrypted traffic.

To maintain a strong security posture, we recommend obtaining a trusted SSL certificate and establishing a process for ongoing monitoring and automated security updates.